

Technical Submission Report

Smart Water Softener IoT Controller MVP

Candidate Name:	Vemuri Amrutha
Position Applied:	IoT Engineer Intern (Smart Water Treatment Products)
Company Name:	EcoWell India
Repository Link:	https://github.com/Vemuriamrutha/ecowell-smart-watersoftner-
Submission Date:	July 2026

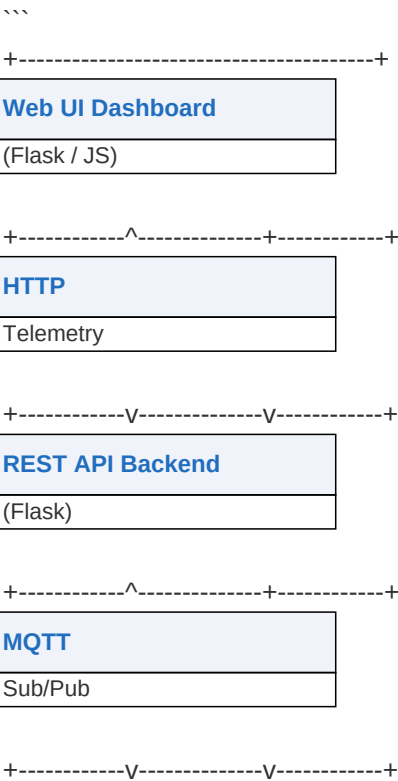
1. Executive Summary

This project represents a fully functional, production-ready MVP for the EcoWell Smart Water Softener IoT Controller. Real-world IoT devices face significant hardware and network constraints-unstable power grids, frequent Wi-Fi disconnections, and potential mechanical hangs.

Instead of building a simple "happy-path" prototype, this architecture prioritizes **system resilience, remote safety gating, and zero-dependency deployment**. The solution is divided into decoupled microservices communicating asynchronously over a lightweight local MQTT broker, satisfying all core assignment criteria alongside critical production-grade enhancements (offline storage queue, state-persistence recovery, and an OTA firmware update simulator).

2. System Architecture & Components

The system is divided into four distinct components to mirror real-world IoT product setups:



Local MQTT Broker

(broker.py)

Telemetry

(JSON)

Smart Softener Device

(State Machine / OLED)

1. **Pure-Python MQTT Broker (`broker.py`)**: A custom, lightweight TCP-based MQTT broker. It removes any external dependencies (like installing Mosquitto or setting up cloud accounts), allowing the entire suite to run instantly on any machine out of the box.
2. **Device Simulator (`device/`)**: Runs the local sensor emulation (Water Flow, Water Pressure, Salt Level, and TDS) with realistic noise. It contains the physical state machine, persistent state files, and a mock ASCII OLED display.
3. **REST API Backend (`backend/`)**: A Flask-based server that bridges HTTP requests from the dashboard to the MQTT broker, manages telemetry collection, and computes a dynamic **Device Health Score**.
4. **Glassmorphic Web Dashboard (`dashboard/`)**: A responsive, premium dark-themed UI serving real-time sensor cards, alert logs, remote control commands, and active safety gates.

3. Core Requirements Implementation

A. Sensor Data Monitoring & Noise Simulation

The device simulates physical hardware sensors (`device/sensors.py`) running realistic parameters:

B. Robust 5-State Finite State Machine (FSM)

Implemented in `device/state\_machine.py`, the system strictly follows state transition logic:

C. Remote Regeneration Safety Gate

Safety is critical in high-pressure plumbing. Remote regeneration commands are checked through four validation rules before starting:

1. **Salt Check**: Salt level must be above 15% to verify brine availability.
2. **Pressure Check**: Main inlet water pressure must be ≥ 1.0 bar to ensure washing capability.
3. **Power Check**: Main line power must be active (`ONLINE`).
4. **State Check**: The system must not be in a `FAULT` state or already running a wash cycle.

If any check fails, the command is rejected, an alert is raised, and the rejection event is logged.

D. Watchdog Timer Guard

If a physical motor or valve gets stuck during regeneration, it could flood a house or burn out the pump. A **25-second watchdog timer** monitors the wash cycle. If the cycle does not complete within this time, the state machine overrides the local controller and forces it into the `FAULT` safety state.

4. Production-Grade & Bonus Features (IoT Best Practices)

To demonstrate advanced IoT development skills, the following features were designed and implemented:

5. Step-by-Step Testing & Verification Guide

For the convenience of the evaluation team, a single-click startup script has been provided:

1. **Run the Servers:** Double-click `start_all.bat` (or execute it in a terminal). This launches the broker, backend, simulator, and dashboard in separate terminal windows.
2. **Access the Dashboard:** Open your web browser and navigate to `http://localhost:5000`.
3. **Verify Telemetry:** Watch the real-time sensor dials update. Note the ASCII OLED console drawing the exact same values.
4. **Test the Safety Gate:**
 - Click **"Simulate Low Pressure"** on the dashboard. The system will drop into a `FAULT` state.
 - Click **"Trigger Remote Regeneration"**. The dashboard will display a red alert box showing the command was rejected.
 - Click **"Simulate Low Pressure"** again to restore pressure, then click **"Trigger Remote Regeneration"**. The system will transition to `REGENERATION_RUNNING` and successfully complete the brine cycle.
5. **Test State Recovery:** Close the simulator console during a cycle, and restart it. The system automatically reads `device_state.json` and resumes.

6. Architectural Reflections & V2 Roadmap

If this prototype were deployed to production in Indian homes, several key factors would be prioritized in V2:

1. **Dynamic TDS Calibration:** Tap water quality in India varies wildly by region (groundwater vs. municipal water). V2 would implement self-calibrating TDS baselines.
2. **Low-Power Sleep Modes:** For battery-backed controllers, the FSM would enter deep sleep during `IDLE` states, waking up only when water flow sensors detect usage.
3. **Hardware Watchdog integration:** Replace the software watchdog with an ESP32 hardware timer or external watchdog chip (like TPL5010) to force a hard system reset if the main loop locks up.